

NAME : S. BHAVYA
REG NO : 192524376

COURSE CODE : CSA0313
COURSE NAME : DATA STRUCTURE

C05-AT1

PSEUDOCODE WRITING TASK

1. Delivery System - Shortest Route:

ALGORITHM DeliveryRoute(G, s)

FOR each vertex v DO
 $distance[v] \leftarrow \infty$

END FOR

$distance[s] \leftarrow 0$

PriorityQueue $\leftarrow \{(0, s)\}$

WHILE PriorityQueue is not empty DO

$(d, u) \leftarrow \text{EXTRACT_MIN}()$

 IF $d \neq distance[u]$ THEN

 CONTINUE

 END IF

 FOR each edge (u, v, w) DO

 IF $distance[u] + w < distance[v]$ THEN

$distance[v] \leftarrow distance[u] + w$

$parent[v] \leftarrow u$

 INSERT($distance[v], v$)

 END IF

 END FOR

END WHILE

OUTPUT shortest delivery routes

END

2. Floyd - Marshall for Smart GPS:

FOR $K \leftarrow 1$ TO V DO

FOR $i \leftarrow 1$ TO V DO

FOR $j \leftarrow 1$ TO V DO

IF $D[i][K] + D[K][j] < D[i][j]$

$D[i][j] \leftarrow D[i][K] + D[K][j]$

END IF

END FOR

END FOR

END FOR

FOR $i \leftarrow 1$ TO V DO

IF $D[i][i] < 0$ THEN

OUTPUT "Negative Cycle Detected"

RETURN

END IF

END FOR

OUTPUT D

END

3. Dijkstra Path through Node K :

Run Dijkstra(G, S)

Store distance S and parents

Run Dijkstra(G, K)

Store distance K and parents

IF distance $S[K] = \infty$ OR distance $K[D] = \infty$ THEN

OUTPUT "No path through K "

ELSE

total $\leftarrow$ distance $S[K] +$ distance $K[D]$

Reconstruct $S \rightarrow K$

Reconstruct $K \rightarrow D$

Combine both paths

OUTPUT path and total distance

END IF

END

4. Floyd - Marshall with Negative Cycle Detection:

```
FOR K ← 1 TO V DO
  FOR i ← 1 TO V DO
    FOR j ← 1 TO V DO
       $D[i][j] \leftarrow \min(D[i][j], D[i][K] + D[K][j])$ 
    END FOR
  END FOR
END FOR

FOR i ← 1 TO V DO
  IF  $D[i][i] < 0$  THEN
    OUTPUT "Negative cycle exists"
    RETURN
  END IF
END FOR

OUTPUT D
```

5. Space - Optimized Floyd - Marshall:

Store only one $V \times V$ distance matrix D

```
FOR K ← 1 TO V DO
  FOR i ← 1 TO V DO
    FOR j ← 1 TO V DO
       $D[i][j] \leftarrow \min(D[i][j], D[i][K] + D[K][j])$ 
    END FOR
  END FOR
END FOR

OUTPUT D

END
```

6. MST + DIJKSTRA for Smart Transportation:

MST $\leftarrow$ Kruskal(G)

OUTPUT "Minimum infrastructure network", MST

Create graph G_{MST} using MST edges.

Run Dijkstra(G_{MST}, S)

IF D is reachable THEN

Reconstruct $S \rightarrow D$

OUTPUT shortest route

ELSE

OUTPUT "Destination Unreachable"

END IF

END

7. Maximum Spanning Tree Using Kruskal:

MST $\leftarrow$ empty set

Create DSU

Sort all edges in Decreasing order of weight

FOR each edge (u, v, w) DO

IF $\text{FIND}(u) \neq \text{FIND}(v)$ THEN

ADD edge to MST

UNION(u, v)

END IF

IF $\text{Size}(\text{MST}) = V - 1$ THEN

BREAK

END IF

END FOR

OUTPUT MST and maximum reliability score

END

8. Critical Dependencies:

```
originalOrder  $\leftarrow$  TopologicalSort( $G$ )
critical  $\leftarrow$  empty list
FOR each edge  $e(u, v)$  in  $G$  DO
    temporarily remove  $e$ 
    newOrder  $\leftarrow$  TopologicalSort( $G$ )
    IF newOrder  $\neq$  originalOrder THEN
        AND  $e$  to critical
    END IF
    Restore  $e$ 
END FOR
OUTPUT critical
END
```

9. Topological Sort on a Forest:

```
Find connected DAG components
Create one queue for each component
Calculate indegree for every task
WHILE at least one queue is non-empty DO
    FOR each component  $C$  DO
        IF  $C$ 's queue is not empty THEN
            Select a zero-indegree task
            OUTPUT task
            Remove task from
            Updates indegrees of its neighbors
            Add newly zero-indegree tasks to  $C$ 's queue
        END IF
    END FOR
END WHILE
END
```


10. BFS - Exactly 3 Segments of Separation:

distance $[u] \leftarrow 0$

visited $[u] \leftarrow \text{TRUE}$

Queue $\leftarrow \{u\}$

WHILE Queue not empty DO

$u \leftarrow \text{DEQUEUE}(\text{Queue})$

 IF distance $[u] = 3$ THEN

 CONTINUE

 END IF

 FOR each v in $\text{adj}[u]$ DO

 IF visited $[v] = \text{FALSE}$ THEN

 visited $[v] \leftarrow \text{TRUE}$

 distance $[v] \leftarrow \text{distance}[u] + 1$

 ENQUEUE (Queue, v)

 END IF

 END FOR

END WHILE

OUTPUT all v where distance $[v] = 3$

END